

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP.HCM

Khoa Công Nghệ Điện Tử - Bộ môn Điện Tử Máy Tính

BÀI GIẢNG

**HỆ THỐNG NHÚNG
(Embedded System)**

TP.HCM-2024

NỘI DUNG MÔN HỌC

Chương 1: GIỚI THIỆU HỆ THỐNG NHÚNG

Chương 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

Chương 3: KỸ THUẬT PHẦN MỀM

Chương 4: THIẾT KẾ HỆ THỐNG NHÚNG

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

1. NGÔN NGỮ LẬP TRÌNH
2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG
3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

Contents

- ☐ Flowcharts
- ☐ Algorithmic State Machine charts (ASM)
- ☐ Timing calculation

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.1 Flowcharts

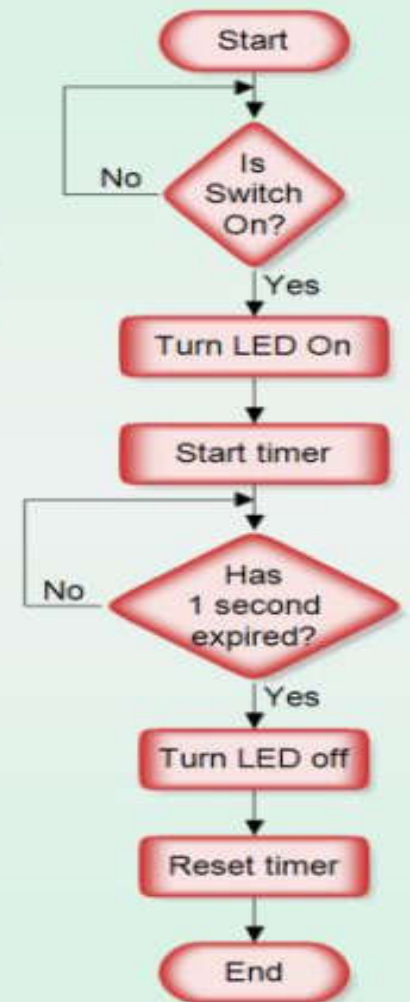
- + **Flowcharts** is a tool **for precise description** of algorithms/procedures
- + **Specify tasks to perform and their sequencing.**
- + **Main symbols:**
 - **Operation box:** contains tasks/operations to perform
 - **Decision box:** alternative actions based on decisions to be taken
 - **Arrows:** indicate appropriate sequencing
- + **Iteration:** some tasks/operations may be repeatedly/iteratively done

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.1 Flowcharts

- **Example:** LED on for 1(second)
as response to a switch event.

Flowchart of a
One Second
Response
to a Switch



3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.2 Algorithmic State Machine (ASM) charts

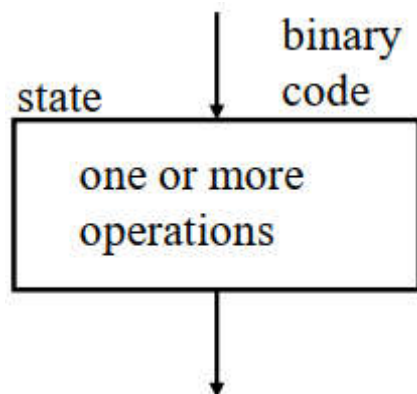
- + **ASM chart**: is a popular **high-level flowchart** - like **graphical model** to specify the algorithms in digital systems.
- + **Major differences** from **flowcharts** are:
 - Uses 3 types of boxes: **state box** (similar to operation box), **decision box**, and **conditional box**.
 - Contains **exact timing information**.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

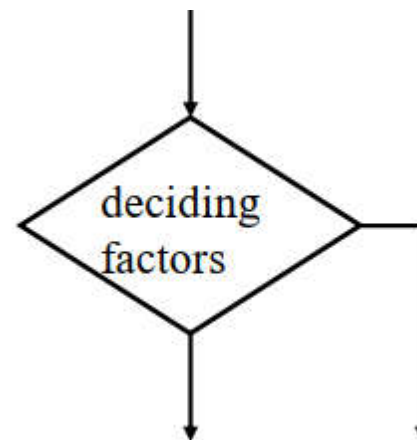
3.2 Algorithmic State Machine (ASM) charts

□ Components of ASM charts:

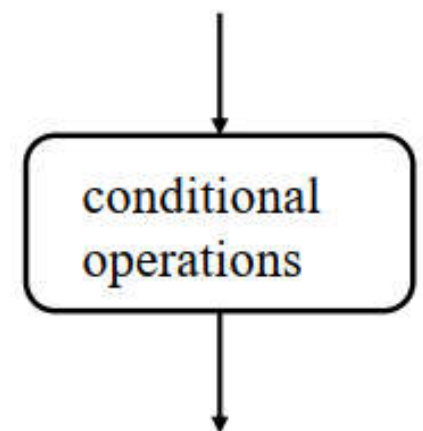
- **State box:** (is used to specify **one** or **more operations** which could be simultaneously completed in **one clock cycle**.)



- **Decision box:**



- **Conditional box:**



3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.2 Algorithmic State Machine (ASM) charts

□ Real-Time operation in an ASM chart

- **RT operation** in an ASM chart is controlled by an embedded **clock signal**.
- **Destination register** is updated when **the machine exits** the **current ASM block**, but **not within** the block.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.3 Timing calculation

□ Timing in ASM charts

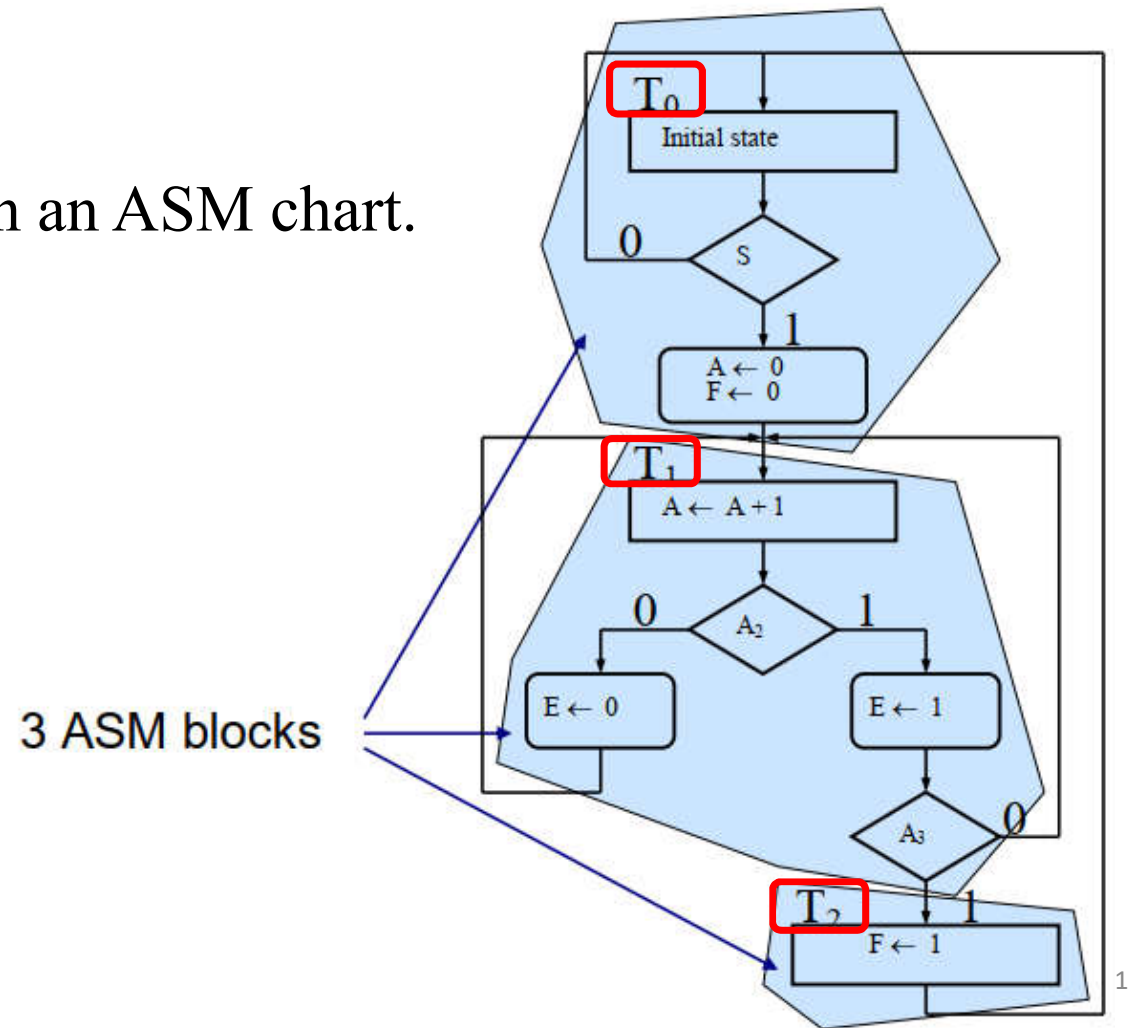
- In the context of designing **Sequential Digital Circuits**: → **precise timing** is **implicitly present** in ASM charts.
- Each (**state box**) or (**decision/conditional box**) occurs **within one clock cycle**.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.3 Timing calculation

Example: Timing calculation in an ASM chart.

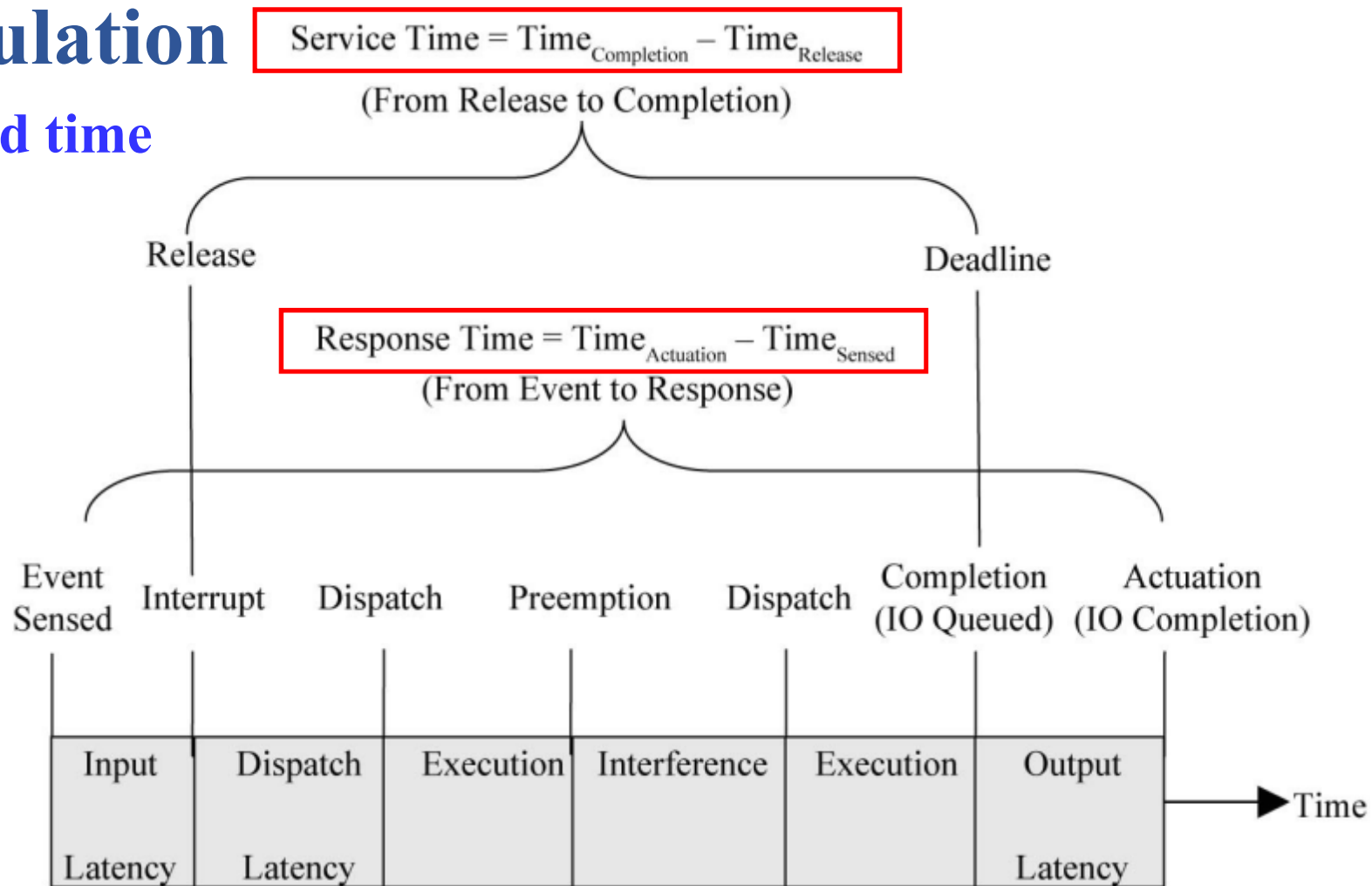


3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.3 Timing calculation

- Service and Respond time



3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.3 Timing calculation

- All services in a **hard real-time system** must have response times less than their required deadline, and the response time must be assumed to be the sum of the total **Worst-Case latency**.
- **Respond time:** $\forall S_i, T_{response(i)} \leq Deadline_i, S_j \subset S_i, where Priority_j > Priority_i$

$$T_{response(i)} = T_{IO_Latency(i)} + WCET_i + T_{Memory_Latency(i)} + T_{interference(j)}$$

$$WCET \equiv Worst_Case_Execution_Time$$

$$T_{interference(j)} \equiv T_{response(i)} - \sum_{j=1}^{i-1} Total_Preempt_Time_{(j)}$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.3 Timing calculation

- **Estimated WCET.** Our estimated WCET is obtained by **analyzing the program code**.
- **Static analysis** method for **estimating execution time** may be performed **on the object code**.
 - By construction, the **static analysis** will produce a **Safe over-estimate** - that is, it will produce an estimate that is **greater than** the **actual WCET**, but **never lesser**.
 - Furthermore, the **estimation in static analysis** is also tight - that is, it is **as close** to the **actual WCET** as possible.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.3 Timing calculation

- ❑ **WCET Calculation.** We only concentrate on how to **subdivide** a program into pieces and how to compose the **WCET estimates** of the **pieces** (to get the WCET estimate of the **whole program**).
- Because with each task (which is actually **a program**), the WCET estimate will be obtained **offline** (by *static analysis*) and provided to schedulability analysis as input.
- Basically, the estimation method is called the *timing schema*.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.3 Timing calculation

- **The timing schema approach.** To estimate the WCET of a statement, some of the prominent rules for **sequential composition**, **if-statement** and **while-statement** are as follows:

■ **Time(S1; S2)**

$$= \text{Time}(\text{S1}) + \text{Time}(\text{S2})$$

■ **Time(if B {S1} else {S2})**

$$= \text{Time}(\text{B}) + \max(\text{Time}(\text{S1}), \text{Time}(\text{S2}))$$

■ **Time(while B {S})**

$$= (\mathbf{n} + 1) * \text{Time}(\text{B}) + \mathbf{n} * \text{Time}(\text{S}) \quad (\text{where } \mathbf{n} \text{ is the loop bound})$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.3 Timing calculation

■ $\text{Time}(\text{for}(\text{Init}; B; \text{Inc})\{ S \})$

$$= \text{Time}(\text{Init}) + (n + 1) * \text{Time}(B) + n * (\text{Time}(S) + \text{Time}(\text{Inc}))$$

■ $\text{Time}(\text{if } B \{ S \}) = \text{Time}(B) + \text{Time}(S)$

- For simplicity, let us assume for now that every *assignment*, *condition*, and *control transfer statement* is **constant** and takes **one time unit**.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.3 Timing calculation

Example: Estimate the **WCET** of the program fragment given in Figure 3.2.

- For convenience we refer to the first if-statement **if (i % 2 == 0)**.. as **if1**.
- And the second if-statement **if (sum < 0)**... as **if2**.
- We also call the **for-loop** in the code-fragment as **for1**:

```
sum=0;
for (i=0; i<10; i++){
    if (i%2==0)
        sum+=i;
    if (sum<0)
        break;
}
return sum;
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.3 Timing calculation

○ Now, let us estimate the WCET of the program fragment given in [Example](#).

■ Time([if2](#))

$$= \text{Time}(\text{if} (\text{sum} < 0) \text{ break}) = \text{Time}(\text{sum} < 0) + \text{Time}(\text{break}) = 1 + 1 = 2 \text{ (Time Units)}.$$

■ Time([if1](#)) = $\text{Time}(\text{if} (i \% 2 == 0) \text{ sum} += i) = \text{Time}(i \% 2 == 0) + \text{Time}(\text{sum} += i) = 2TU.$

■ Time([for1](#)) = $\text{Time}(i=0) + 11 * \text{Time}(i < 10) + 10 * (\text{Time}(\text{if1}; \text{if2}) + \text{Time}(i++))$
 $= 1 + 11 * 1 + 10 * (\text{Time}(\text{if1}) + \text{Time}(\text{if2}) + 1) = 1 + 11 + 10 * (2 + 2 + 1) = 62 TU.$

■ Time(Program of [Figure 3.2](#))

$$= \text{Time}(\text{sum}=0) + \text{Time}(\text{for1}) + \text{Time}(\text{return sum}) = 1 + 62 + 1 = \textbf{64 time units.}$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

□ CPU Utilization factor

- The CPU utilization or time-loading factor is a **critical measure** of real-time system **performance**.
 - The CPU **have** more or less frequently execute instructions **that are not related** to the **completion** of a **specific deadline**.
 - The measure of the **relative time** spent **doing non-idle processing** **indicates** how much real-time processing is occurring.
- The CPU utilization or time-loading factor, **U**, is a **relative measure** of the **non-idle processing** taking place.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

□ CPU Utilization factor

- U is calculated by **summing** the **contribution of** utilization factors for each task.
- A system has $n \geq 1$ **periodic tasks**, each with an **execution period** of p_i
- If **task i** is known to have a **worst-case execution time** of e_i , then the utilization factor, u_i , for **task i** is:

$$u_i = e_i / p_i \quad (1.1)$$

- Furthermore, the **Overall system utilization factor** is:

$$U = \sum_{i=1}^n u_i = \sum_{i=1}^n e_i / p_i \quad (1.2)$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

Utilization %	Zone Type	Typical Application
< 26	Unnecessarily safe	Various
26 – 50	Very safe	Various
51 – 68	Safe	Various
69	Theoretical limit	Embedded systems
70 – 82	Questionable	Embedded systems
83 – 99	Dangerous	Embedded systems
100	Critical	Marginally stressed system
> 100	Overloaded	Stressed system

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

❑ Performance Analysis

a/ Polled-Loop System: continuously checks (or polls) the status of events in a loop without waiting for interrupts.

- **Response time** = Time to check all conditions per loop cycle.

```
while (1)
{
    if (sensor_ready())
        read_sensor()

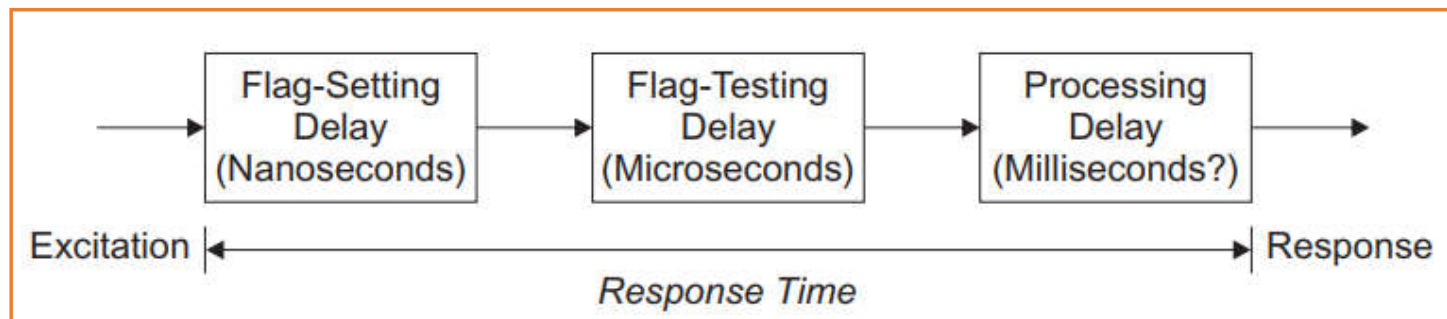
    if (button_pressed())
        handle_button()

    ....
}
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- **The response time** consists of three essential components (**see Fig**):
 - The **cumulative hardware delays**
(involved in setting the software flag by some external device).
 - The **time for the polled loop** to *test the flag*.
 - The **time needed to process the event** *associated with the flag*.



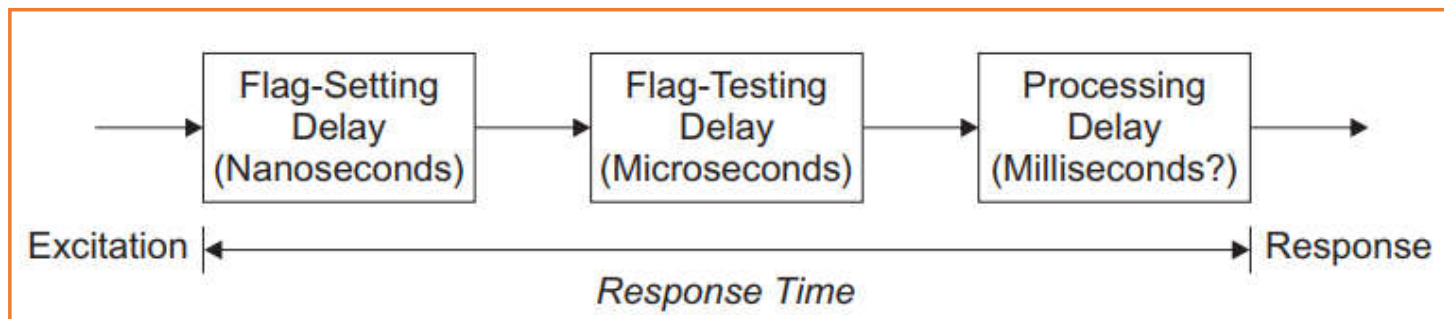
3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

○ **Response time:**

$$Bound = N * (t_F + t_P)$$

- t_F is the time needed to check the flag,
- t_P is the time to process the event,
- N , is the number of events that are allowed to overlap.



3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Timing calculation

b/ Coroutine System: allow multiple tasks to run cooperatively by **yielding** control explicitly (**not via preemption**).

○ **Maximum response time =**

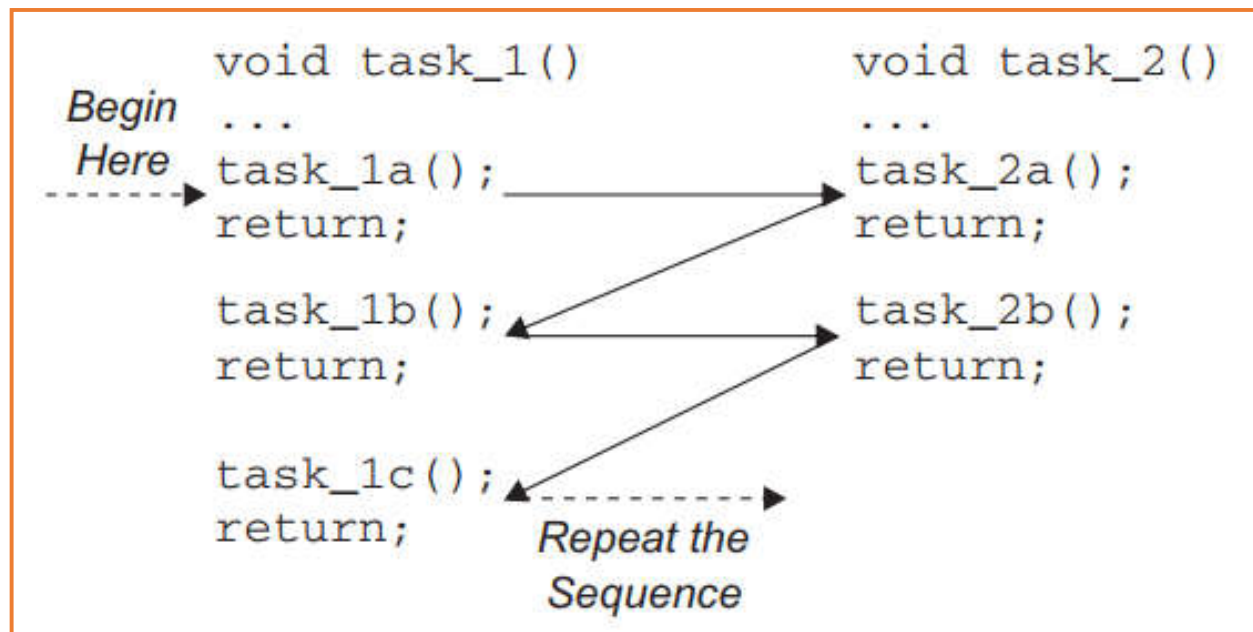
Sum of execution time of all tasks per round + **scheduler overhead** (**Negligible**=very small).

```
def task1():  
    while True: read_sensor()  
                yield  
  
def task2():  
    while True: handle_output()  
                yield  
  
# Cooperative Scheduler  
while True:  
    next (task1_gen)  
    next (task2_gen)
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- **Response time:** the absence of interrupts in a **Coroutine system** makes the determination of response time **rather easy**, and the time is simply obtained by tracing the **worst-case execution path through all tasks**.



3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

c/ Round Robin System: is **scheduling algorithm**. In this system, each task can be provided a **time slice** (*quantum*) by the CPU. After that slice, CPU will continue implement the next task in round robin.

□ Assume that a Round-robin system has **n tasks** (τ_i) in the ready queue.

- each task, τ_i , has a different maximum execution time, c_i ; then letting
→ $c = \max\{c_1, \dots, c_n\}$ is **upper bound**=the maximum execution time.
- let the **constant time slice** for each task be q time units.
- each task still waits **no longer than** $(n-1)q$ until its **next time quantum**
- each task requires at most $\lceil c/q \rceil$ time quanta **to complete**.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

→ the waiting time will be: $(n - 1)q.[c/q]$

○ Thus, **the worst-case time** to completion for any task (**Turnaround time**):

→ denoted **T** , is the **waiting time** + **time to complete**, **c**

$$T = (n - 1)q.[c/q] + c \quad (1.3)$$

Example 1: Turnaround time calculation without Context switching **overhead**

- **First**, there is **only 1 task** **$n=1$** with a maximum execution time of **$c= 500$ ms**, and that the time quantum is **$q= 100$ ms**.

$$T = (n - 1)q*[c/q] + c$$

$$\rightarrow T = (1 - 1)100*[500/100] + 500 = \mathbf{500 \text{ ms.}}$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- **Next**, suppose there are **five equally important tasks** $n=5$, with a maximum execution time of $c= 500$ ms. The time quantum is **still** $q= 100$ ms.

$$\rightarrow T= (5 - 1)100*[500/100] + 500 = \mathbf{2500\ ms}$$

- This result is **intuitively agreeable**, since it would be expected that **five consecutive tasks of 500 ms**, each would take altogether **2500 ms** end-to-end to complete.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- Assume that there is a **non-negligible context switching overhead**, o , associated with **task switching**.
- An additional assumption is that there is an initial “context switch” to load the first time around. Therefore, the **worst-case turnaround time** for any task is now at most.

$$T = [(n - 1)q + n.o].\lceil c/q \rceil + c \quad (1.4)$$

Example 2: Turnaround-time calculation with context switching Overhead

- **First**, there is one task with a maximum execution time of $c = 500$ ms. The time quantum is now $q = 40$ ms, the **context switch time** is $o = 1$ ms.
→ $T = [(1 - 1)40 + 1.1] * \lceil 500/40 \rceil + 500 = 513$ ms.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- **Next**, suppose there are **six equally important tasks $n=6$** , each with a maximum execution time of **$c = 600$ ms**, **$q=40$ ms**, and every context switch costs **$o=2$ ms**. Then

$$\rightarrow T = [(6 - 1)40 + 6.2] * [600/40] + 600 = 3780 \text{ ms.}$$

which **again is agreeable**, because one would **expect six tasks of 600 ms** in duration to already **take 3600 ms without** any context switching costs.

- In general, **simple utilization-based analysis** is not accurate and provides satisfactory bounds just for a highly simplified task model. Therefore, **a necessary and sufficient condition** for schedulability based on **worst-case response-time calculation** is presented below.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

d/ Fixed-Period Systems

- For the **highest-priority task**: the worst-case response time will **be equal** to its own execution time. **Other tasks** running on the real-time system **are subjected to interference** caused by **execution of higher-priority tasks**.
- For any task **τ_i** with an execution time of **e_i** time units, the **response time, R_i** , is given as:

$$R_i = e_i + I_i \quad (1.5)$$

where **I_i** is the **maximum possible delay** in execution (caused by higher priority tasks)

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Timing calculation

□ Consider a task τ_j of **higher priority** than τ_i .

- Within the interval $[0, R_i) \rightarrow$ the **release time** of τ_j will be $\lceil R_i / p_j \rceil$,

where p_j is the **execution period** of τ_j .

- **Each release of task τ_j** is going to contribute to the **amount of interference τ_i** suffered, and is expressed as:

$$\text{Maximum interference} = \lceil R_i / p_j \rceil e_j$$

- **Each task of higher priority** is **interfering** with task τ_i . Hence,

$$I_i = \sum_{j \in HPR(i)} \lceil R_i / p_j \rceil e_j, \text{ where } HPR(i) \text{ is the set of higher-priority tasks with respect to } \tau_i.^{34}$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- Substituting this I_i into Equation 1.6 yields

$$R_i = e_i + \sum_{j \in HPR(i)} \left\lceil R_i / p_j \right\rceil e_j \quad (1.6)$$

- With the inconvenient **Ceiling function**, it is difficult to solve for R_i directly. Therefore, a recursive solution is provided, where the equation for calculating R_i is evaluated iteratively as a recurrence relation.

$$R_i^{n+1} = e_i + \sum_{j \in HPR(i)} \left\lceil R_i^n / p_j \right\rceil e_j \quad (1.7)$$

where R_i^n is the result of the n -th iteration.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

- When using the **recurrence relation** to find **response times**, it is necessary to compute **consecutive values** of R_i^{n+1} iteratively until the first value of m is found such that $R_i^{m+1} = R_i^m$.
→ This R_i^m is then **the desired response time, R_i** .
- It is important to note that → if the recursive equation **does not have a solution**,
→ Then the value of R_i^{n+1} will **continue to grow**, as in the **overloaded case** when a task set has a **CPU utilization factor** greater than 100%.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

Example 3: Response Time Calculation in a Rate-Monotonic Case

- Consider a task set to be scheduled rate monotonically, as shown below:

(highest priority task) τ_1 : $e_1 = 3, p_1 = 9$; τ_2 : $e_2 = 4, p_2 = 12$; τ_3 : $e_3 = 2, p_3 = 18$

- For every task set, we need to calculate first the **CPU utilization factor**, U of Equation 1.2, to **make sure** that the real-time system is **not overloaded**. Here:

$$U = \sum_{i=1}^3 [e_i / p_i] = \frac{3}{9} + \frac{4}{12} + \frac{2}{18} \approx 0.72$$

→ 72% belongs to the “questionable” utilization zone of **70–82%**, which is well **below overloading**.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

(highest priority) τ_1 : $e_1 = 3, p_1 = 9$; τ_2 : $e_2 = 4, p_2 = 12$; τ_3 : $e_3 = 2, p_3 = 18$

- The highest priority task, τ_1 , will naturally have a response time equal to its execution time, so $R_1 = 3$.
- For the *medium priority* task τ_2 , using **Equation 1.7** for its response time. **First**, let $R_2^0 = 4$, and then two recursive values following are :

$$R_2^1 = 4 + \lceil 4 / 9 \rceil \cdot 3 = 7$$

$$R_2^2 = 4 + \lceil 7 / 9 \rceil \cdot 3 = 7$$

→ The equality $R_2^1 = R_2^2$ implies that $R_2 = 7$.

$$R_i^{n+1} = e_i + \sum_{j \in HPR(i)} \left\lceil R_i^n / p_j \right\rceil e_j$$

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

(highest priority) τ_1 : $e_1 = 3, p_1 = 9$; τ_2 : $e_2 = 4, p_2 = 12$; τ_3 : $e_3 = 2, p_3 = 18$

- Similarly, the response time of the *lowest priority* task τ_3 , is calculated as follows. First, $R_3^0 = 2$, and two recursive values are again obtained

$$R_3^1 = 2 + \left\lceil \frac{2}{9} \right\rceil \cdot 3 + \left\lceil \frac{2}{12} \right\rceil \cdot 4 = 9$$

$$R_3^2 = 2 + \left\lceil \frac{9}{9} \right\rceil \cdot 3 + \left\lceil \frac{9}{12} \right\rceil \cdot 4 = 9$$

→ The equality $R_3^1 = R_3^2$ implies that $R_3 = 9$.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Timing calculation

Example 4: Response Time Calculation without the context switch time

Task Id.	Task Cycle	Time Needed (ms)	Priority (1 is highest)
τ_1	10 ms	4	1
τ_2	20 ms	5	3
τ_3	40 ms	10	2
τ_4	Background	5	n/a

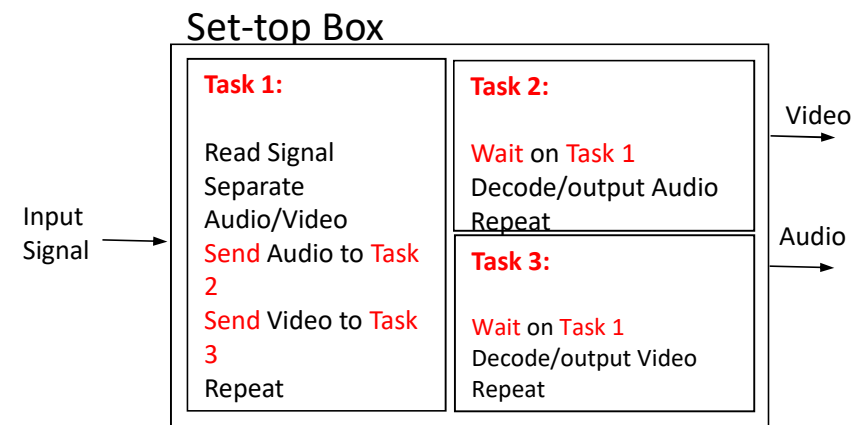
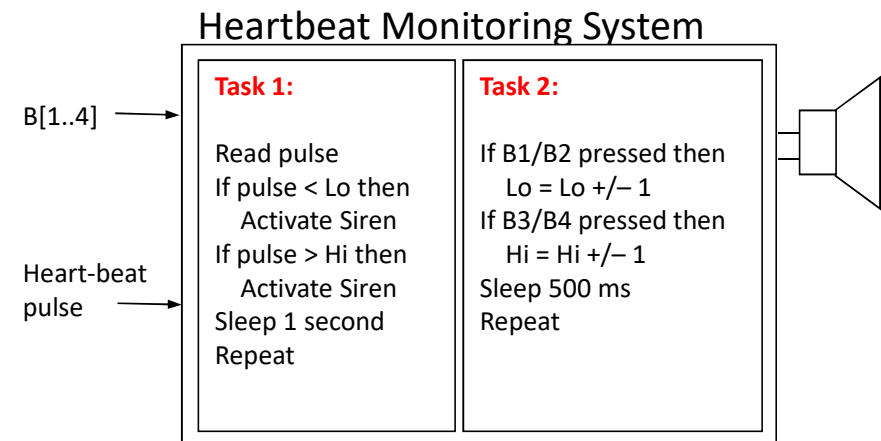
3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Optimization

□ Concurrent Processes

- Consider 2 examples having separate tasks **running independently** but **sharing data**.
- Difficult to write system using sequential program model.
- Concurrent process model** easier:
 - Separate sequential programs (processes) for each task.
 - Programs **communicate** with each other.



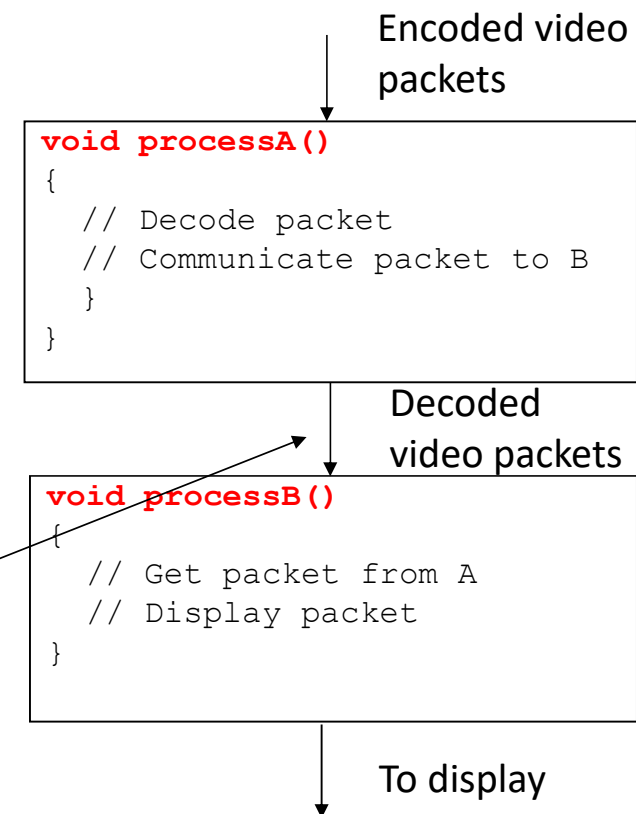
3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Optimization

❑ Communication among Processes

- Processes need to communicate **data and signals** to solve their computation problem.
 - Processes that don't communicate are just independent programs solving separate problems
- Basic example:** producer/consumer
 - Process A produces data items, Process B consumes them
E.g., A decodes video packets, B display decoded packets on a screen.
- How do we **achieve** this communication?
 - Two basic methods: **Shared memory + Message passing**



3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Optimization

▪ Shared Memory

Processes read and write shared variables

- No time overhead, **easy to implement**
- But, **hard to use** – mistakes are common

Example: Producer/consumer with **a mistake**

- Share **buffer[N]**, **count**
(**count** = # of valid data items in **buffer**)
- Process A produces data items and stores in **buffer**
(If **buffer** is **full**, must **wait**)
- Process B consumes data items from **buffer**
(If **buffer** is **empty**, must **wait**)

```
01: data_type buffer[N];
02: int count = 0;
03: void processA() {
04:     int i;
05:     while( 1 ) {
06:         produce(&data);
07:         while( count == N ); /*loop*/
08:         buffer[i] = data;
09:         i = (i + 1) % N;
10:         count = count + 1;
11:     }
12: }
13: void processB() {
14:     int i;
15:     while( 1 ) {
16:         while( count == 0 ); /*loop*/
17:         data = buffer[i];
18:         i = (i + 1) % N;
19:         count = count - 1;
20:         consume(&data);
21:     }
22: }
23: void main() {
24:     create_process(processA);
25:     create_process(processB);
26: }
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Optimization

▪ Message passing

Data explicitly sent from one process to another

- Sending process performs special operation, *send*.
- Receiving process must perform special operation, *receive*, to receive the data.
- **Both operations** must explicitly specify which process it is sending to or receiving from
- *Receive* is blocking, *send* may or may not be blocking.

→ Safer model, but less flexible

```
void processA()  
{  
    while( 1 ) {  
        produce(&data)  
        send(B, &data);  
        /* region 1 */  
        receive(B, &data);  
        consume(&data);  
    }  
}
```

```
void processB()  
{  
    while( 1 ) {  
        receive(A, &data);  
        transform(&data)  
        send(A, &data);  
        /* region 2 */  
    }  
}
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Optimization

□ Loop Optimization

- **Increasing** **execution speed** and **reducing** the **running costs** associated with loops.
- It plays an important role in **improving cache performance** and **making effective use** of **parallel processing capabilities**.
- **Decreasing** the **number of instructions** in an inner loop to improve the **running time** of a program.

```
while (x<10) { y := 3 * x+1; a[y] := a[y]-2; x := x+2; }
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

1. Strength Reduction

- **Replacing** expensive operations with cheaper ones like: **multiplication** is costlier than **addition**, so multiplication can be replaced by addition in the loop. For example:

```
while (x<10)
{
    y := 3 * x+1;
    a[y] := a[y]-2;
    x := x+2;
}
```

could be replaced with:

```
t= 3 * x+1;
while (x<10)
{
    y=t;
    a[y]= a[y]-2;
    x=x+2;
    t=t+6;
}
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

2. Common subexpression elimination

- It should be avoided that repeated calculations of the same subexpression in two different expressions. For example:

```
x=6+a*b;  
y=a*b+z;
```

→ could be replaced with:

```
t=a*b;  
x=6+t;  
y=t+z; →thus eliminating the other multiplication (t=a*b)
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU



CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

3.4 Optimization

3. Use intrinsic functions (rather than ordinary functions)

- **Intrinsic functions** are simply macros where the actual function call is replaced by **inline code** during compilation.
- And although the **original expression** may be more descriptive, a **comment** can be provided to explain the optimized one. For example:
 - if the program uses $\pi/2$, it could be precomputed during the initialization phase and stored as a constant named, ***pi_div_2***.
- This will typically save **one floating point load** and **one floating point divide instruction** - potentially several microseconds.

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

4. Loop invariant removal

- The expression with computation is **avoided inside** the loop. → That computation **is performed outside** the loop as **computing the same expression** each time was overhead to the system → and **this reduces** computation overhead and hence optimizes the code.

```
for (int i=0; i<10;i++)  
    t= i+(x/y);  
    ...  
end;
```

```
s = x/y;  
for (int i=0; i<10;i++)  
    t= i+ s;  
    ...  
end;
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

4. Loop invariant removal

```
x=100;  
while (x>0)  
    x=x-y+z;
```

→ It can be replaced by:

```
x=100;  
t=y+z;  
while (x>0)  
    x=x-t;
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

4. Loop invariant removal

- An integer variable *i* is called an **induction variable of a loop** if it is incremented or decremented by some constant **on each cycle of the loop**.

```
for (i=1;i<=10;i++)  
    a[i+1]=1;
```

→ An optimized version is:

```
for (j=2;j<=11;j++)  
    a[j]=1;
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

5. Loop Unrolling

- This is a **loop transformation technique** that helps to **optimize the execution time** of a program. We basically **remove** or **reduce iterations**.
- Loop unrolling increases the program's speed **by eliminating** loop control instruction and loop test instructions.

for (i=1;i<=6;i++) a[i]=a[i]*8; → is replaced by:

```
a[1]=a[1]*8; a[2]=a[2]*8;  
a[3]=a[3]*8; a[4]=a[4]*8;  
a[5]=a[5]*8; a[6]=a[6]*8;
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

6. Loop Jamming (or Loop Fusion)

- This is a technique for combining two similar loops into one, thus reducing loop overhead by a factor of two. For example:

```
for (i=1;i<=100;i++)  
    x[i]=y[i]*8;  
for (i=1;i<=100;i++)  
    z[i]=x[i]*y[i];
```

```
for (i=1;i<=100;i++)  
{  
    x[i]=y[i]*8;  
    z[i]=x[i]*y[i]; } }
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

7. Loop Fission

- Loop fission **improves** the **locality of reference**, in loop fission a single loop is **divided into multiple loops** over **the same index range**, and each divided loop **contains a particular part** of the **original loop**.

```
for(x=0;x<10;x++)  
{  
    a[x]=...  
    b[x]=...  
}
```

```
for(x=0;x<10;x++)  
    a[x]=...  
for(x=0;x<10;x++)  
    b[x]=...
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

8. Loop Interchange

- In loop interchange, **inner loops** are **exchanged** with **outer loops**. This optimization technique also **improves** the **locality of reference**.

```
for(x=0;x<10;x++)  
  for(y=0;y<10;y++)  
    a[y][x]=...
```

```
for(y=0;y<10;y++)  
  for(x=0;x<10;x++)  
    a[y][x]=...
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

9. Loop Reversal

- Loop reversal reverses **the order of values** that are assigned to index variables. This help in **removing dependencies**.

```
for(x=0;x<10;x++)  
    a[9-x]=...
```

```
for(x=9;x>=0;x--)  
    a[x]=...
```

3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

3.4 Optimization

9. Loop Splitting

- Loop Splitting simplifies a loop **by dividing it into numerous loops**, and all the loops have some bodies but **they will iterate over different index ranges**.
- Loop splitting helps in **reducing dependencies** and hence making code more optimized.

```
for(x=0;x<10;x++)  
  if(x<5)  
    a[x]=...  
  else b[x]=...
```

```
for(x=0;x<5;x++)  
  a[x]=...  
for(;x<10;x++)  
  b[x]=...
```